Muscle BIDS Manual

QMRITools

Manual version: 2.1 · QMRITools version: 5.1 · Date: 2026-11-02 · Author: Martijn Froeling

This manual explains how to process a muscle MRI study with the Muscle BIDS pipeline of QMRITools: how to set up a study, how to write the `config.json` file that drives the pipeline, how to run each step, and how to check the results. You do not need to know Mathematica; every command is a line you copy into a notebook. If you are new, read Muscle BIDS in QMRITools first, with the overview of the functions, and then follow A complete run, step by step.

Contents

Muscle BIDS in QMRITools
- How it works, and why
- Overview of the functions

Before you start

Glossary

Setting up a study
- The source data
- The study folder
- Your config file

Running the pipeline
- 1. BidsDcmToNii
- 2. MuscleBidsConvert
- 3. MuscleBidsProcess
- 4. MuscleBidsMerge
- 5. MuscleBidsSegment
- 6. MuscleBidsTractography
- 7. MuscleBidsAnalysis

A complete run, step by step
- Folder layout after a run
- Starting from an example config

The config file
- Writing the file (JSON)
- The four sections
- What happens when a key is missing
- folders
- conversion

Datasets: describing your scans
- A first dataset
- Data description keys
- Dataset names and the Key
- How the output files are named

Processing, merging, segmentation and tractography
- Process
- Merging
- Segment
- Tractography
- Per-subject settings

Analysis: the analysis section
- Minimal analysis example
- How the section is built
- Segmentation
- Analysis
- Images
- Choosing the pictures: BidsOutputImages
- Output: the Excel files
- Nested analysis blocks

Checking and viewing your study
- ViewConfig
- SelectSubjects
- ViewProtocolNames
- CheckConfigLabels
- CheckBidsOutput
- CheckCustomConfigs
- MuscleBidsConfig
- MuscleBidsCheck
- ManageCheckFiles
- MuscleBidsReport
- MuscleBidsGUI
- How the pipeline skips finished work
- How to force a redo
- Log
- Nothing happened: what to check

Appendix: advanced settings
- Default config
- Pipeline step options
- Dataset options
- Process options
- Merging options
- Segment options
- Tractography options
- Per-subject file names
- Analysis options

Muscle BIDS in QMRITools

Muscle BIDS is the QMRITools way to organise and process muscle MRI studies. It follows the BIDS standard, a common folder and file naming scheme for neuroimaging data. For an introduction to BIDS, see https://bids.neuroimaging.io/index.html. For the full specification, see https://bids-specification.readthedocs.io/en/stable/.

QMRITools puts all your data in one study folder. The pipeline then converts, processes, merges, segments and analyses the data in a fixed order, with one command per step. Each step writes its results into its own folder, so you can always see what was made and where.

How QMRITools names files and folders is described at https://www.qmritools.com/bids/files_folders/. The list of entities and suffixes is at https://www.qmritools.com/bids/ents_suffs/.

> **How to run a command (for people new to Mathematica)**
>
> You type commands in a Mathematica notebook and run them with **Shift+Enter**.
>
> 1. Open Mathematica and create a new notebook. Save it in your study folder, next to `config.json` (the file is explained in The config file; a starting file is in Starting from an example config).
> 2. Load QMRITools by typing `Needs["QMRITools`"]` and pressing Shift+Enter.

3. Set the study folder by typing `dir = NotebookDirectory[];` and pressing Shift+Enter.
4. Run a step by typing its command, for example `MuscleBidsProcess[dir, ProcessSubjects -> All]`, and pressing Shift+Enter.

`ProcessSubjects -> All` runs the step for every subject. To run it for some subjects only, give their names as a list, using the folder names from your source data:

`MuscleBidsProcess[dir, ProcessSubjects -> {"001", "002_2"}]`

A subject name without a session, such as `"001"`, always means session `001`.

How it works, and why

The pipeline is built around one idea: every file has one fixed place and one fixed name, which is made from the subject, the session and the kind of scan. Because of this, no step needs a list of files from you. A step looks at the subject and the session, works out the name of the file it needs, and goes to get it. This is what makes it possible to run the same commands on a study of five subjects or of five hundred. It is also the reason for the rules on names: subject and session names may not contain hyphens, underscores, periods or spaces, because these characters are used to build the file names and to split them again.

You describe your study once, in the config file, and the same program is used for every study. The config says which scans you acquired, what they are called on the scanner, and what should be done with them. The link between your scans and the config is the protocol name that the scanner wrote into the data. A dataset in the config lists the protocol names that belong to it, and that is how the pipeline knows which files to pick up. For this reason the names in the config must match the names on the scanner exactly.

The steps form a chain. Each step reads the folder that the step before it wrote and writes into its own folder. Nothing is kept in memory in between: the folders on disk are the only thing the steps share. This has practical consequences. You can stop after any step and look at what it made, you can fix a problem and run only that step again, and you can run the next step a week later without anything from before still being open. The price is that a step relies on the names that the earlier steps gave to the files, so the folders and files should not be renamed by hand.

Every subject and session is handled on its own. If one subject fails, for example because a scan is missing, the step reports it and goes on with the next subject. New subjects can be added at any time and processed alone, and the subjects that are already done are not touched.

Processing takes time, so finished work is skipped. When a step has finished a subject, it writes a small check file next to the result. The next time the step runs, it finds that file and moves on. Running the whole chain again is therefore safe, and it only does the new work. The other side of this is that a step does not notice by itself that you changed the config. To redo a step, you delete its check files, and those of the later steps, because they depend on its results. The windows described in Checking and viewing your study show these files and can delete them.

A study design can have exceptions. A subject can have its own small config file that changes the datasets for that subject only, for example when one participant was scanned with a different protocol (see Per-subject settings). Use this for real exceptions. Every exception is something you have to keep track of, and the pipeline is at its best when there are few of them.

Automation only works when the data is consistent. The config holds one set of rules for the whole study, and the pipeline can only apply those rules to data that follows them. Think about the design before the first scan is acquired: choose the protocol names and the subject and session names, and keep them the same for all subjects and visits. This costs some time at the start. It saves much more later, because with inconsistent data you spend the time on exceptions and on repairing the data by hand instead. Most of the cases where “nothing happened” come from a mismatch between the config and the data, such as a label that does not match the protocol name, or a file that is not where the step expects it. Every subject has a log file for each step that says what was done. The check windows and the problem report exist to find these mismatches quickly.

Overview of the functions

Everything in Muscle BIDS is a function that you can call from a notebook. The table lists them in the order in which you normally use them, from the whole study down to the single steps. Each name links to its section.

Stage	Function	What it is for
Everything in one window	`MuscleBidsGUI`	A window to run the steps and to look at the config and the checks. It uses the functions below.
Set up and check the config (before processing)	`MuscleBidsConfig`	The four tools below as tabs in one window.
	`ViewConfig`	Shows the full config.
	`CheckCustomConfigs`	Shows the subjects that have a per-subject config, what it changes, and whether the steps used it.
	`CheckConfigLabels`	Checks that the labels in the config are present in the data.
	`ViewProtocolNames`	Shows the protocol names that the scanner wrote into the data, to write the labels.
Run the pipeline (in this order)	`BidsDcmToNii`	Converts the DICOM data to NIfTI.
	`MuscleBidsConvert`	Sorts and names the files in the BIDS way.
	`MuscleBidsProcess`	Calculates the parameter maps.

Stage	Function	What it is for
	`MuscleBidsMerge`	Joins the stacks and registers them to one space.
	`MuscleBidsSegment`	Segments the muscles.
	`MuscleBidsTractography`	Calculates the muscle fibre tracts.
	`MuscleBidsAnalysis`	Makes the Excel sheets and images.
Check the processing (after or between steps)	`MuscleBidsCheck`	The tools below as tabs in one window, with the report.
	`CheckBidsOutput`	Checks that the converted files are present.
	`ManageCheckFiles`	Shows which steps are finished for each subject, and removes finished marks to redo a step.
Report	`MuscleBidsReport`	Writes the problems of one step, or of all steps, as a file.

The normal order is the order of the table. You write the config and check it, you run the steps, you check what they made, and when you want an overview you make the report.

The GUI is a convenience, not a requirement. Everything that the GUI can do can also be done with the normal functions, and every function can be used without the GUI. The buttons of the **Run** tab start the same steps that you can type, and the **Config** and **Check** tabs are the functions `MuscleBidsConfig` and `MuscleBidsCheck`. The options that the GUI shows as checkboxes and menus are the same options that you give to the functions. This means that you can run a study from a script, on a computer without a screen, or in a notebook that you keep next to your data as a record of what you did. Use the GUI when you want the buttons, and the functions when you want to repeat or automate the work.

Before you start

- **Wolfram Mathematica 15.0 or newer** with **QMRITools** installed (see https://www.qmritools.com). The converter dcm2niix and the segmentation networks come with QMRITools; you do not install them yourself.
- **Your scanner data in DICOM format**, one folder per subject (see The source data).
- **A plain text editor** to write `config.json` (Notepad, VS Code; not Word).
- A folder for the study. In Mathematica, write folder paths with forward slashes, for example `C:/Data/MyStudy`.

Glossary

Term	Meaning
Study folder (`dir`)	The folder that holds `config.json` and the data folders. In the commands of this manual it is the variable `dir`.
Subject, session	One person, and one visit of that person. Folder names are `subject` or `subject_session`.
`ProcessSubjects`	The option that selects which subjects a step runs on: `All` or a list of names.
Protocol name	The name of a scan as set on the scanner. It is stored in the converted `.json` files and shown by `ViewProtocolNames`.
Dataset	One named entry in the `datasets` section of the config: a type of scan you want to process, for example the Dixon scans.
`Label`	In a dataset: the protocol name(s) of the scan(s) that belong to it. In the analysis, *muscle labels* are the numbers in a segmentation, and a label file links each number to a muscle name.
`Type`, `Suffix`	The kind of scan (`megre`, `dwi`, `mese`, ...) and a short word that marks the data in the file names (`dix`, `dti`, `t2`, ...).
`Class`	How several scans in one dataset belong together, for example as stacks.
Stack (chunk)	One part of a muscle group that is scanned separately because it does not fit in one field of view. The merge step joins them.
`Key`	The dataset name as it is used in file names and in `Target` lists.
`Target`	The name of one image, written as a list of parts, for example `["megre", "dix", "outph"]` (type, suffix, map). It starts with the dataset name when needed.
Map	An image calculated from the scans, for example fat fraction (`fatfr`) or water (`wat`).
Native space	The position and resolution of the scan itself, before registration.
Registration	Moving an image so that it lines up with another image.
Check file	A small file ending in `_check.json` that marks a step as finished for a subject, so the step is skipped next time.

Term	Meaning
IQR	Interquartile range: the spread of the values in a muscle, reported next to the median.

Setting up a study

A study folder holds your data and a configuration file. The configuration file tells QMRITools what data to expect, what to convert and how to process it. You can use the pipeline with existing data, but it works best when you plan the processing together with the study design. Consistent data and folder names make the automated processing much more reliable.

If your data is not well structured, converting it from DICOM to BIDS takes manual work. Once the data is in the BIDS structure, the pipeline usually runs without problems. When it crashes, the cause is most often a mistake in the configuration file.

The source data

Your scanner exports data as DICOM files. Put each subject in its own folder inside the source data folder (`01_sourcedata`). Name each folder with one of these formats:

- `subjectName`
- `subjectName_sessionName`

If you do not add a session name, QMRITools uses session `001` .

Do not use hyphens (`-`), underscores (`_`), periods (`.`) or spaces in subject or session names. The underscore is only used to separate the subject from the session.

To see which subjects and sessions QMRITools will find, run `SelectSubjects[dir]` . It shows the folders in a list with checkboxes. Tick the ones you want and press "Copy selected list to clipboard". You can then paste the list as the value of `ProcessSubjects` .

The study folder

The study folder `dir` contains these folders (the default names). QMRITools creates the ones it needs.

Folder	Contents
`01_sourcedata`	The DICOM files from the scanner, one folder per subject and session
`02_rawdata`	Converted NIfTI files with BIDS names, sorted by subject and session
`03_derivatives`	Processed maps (for example fat fraction and diffusion maps), in the native space of the scan
`04_merged`	Joined and registered data, segmentations and tractography results
`05_analysis`	Excel sheets with the results per subject and one combined file for all subjects (plus `.wxf` files, Wolfram data files you can ignore)

The file `config.json` goes in the root of the study folder, next to these folders.

Your config file

The file `config.json` describes your study. It has these parts:

- `folders` (optional): the names of the folders in the table above. Leave it out to use the default names.
- `conversion` (optional): settings for the DICOM conversion. Leave it out to use the defaults.
- `datasets` : the list of data types you acquired, such as Dixon, diffusion (DTI) or T2 mapping. For each one you give the protocol name(s) from the scanner (`Label`), the kind of data (`Type`), and how to process, merge, segment and track it.
- `analysis` : the parameters and images you want from the analysis.

To look at the file in a readable form, run `ViewConfig[dir]` . All settings are explained in The config file and the chapters after it.

Running the pipeline

The pipeline has seven steps. Run them in this order. Each step reads the output of the step before it.

Step	Group	What it does
`BidsDcmToNii`	Conversion	Converts DICOM to NIfTI
`MuscleBidsConvert`	Conversion	Sorts and names the NIfTI files in BIDS format
`MuscleBidsProcess`	Processing	Calculates the parameter maps for each data type
`MuscleBidsMerge`	Post-processing	Joins the stacks of each muscle group and registers them to one common space

Step	Group	What it does
`MuscleBidsSegment`	Post-processing	Segments the muscles automatically
`MuscleBidsTractography`	Post-processing	Calculates the muscle fibre tracts from diffusion data
`MuscleBidsAnalysis`	Analysis	Summarises the results in Excel sheets and makes images

Steps 3 to 7 skip work that is already finished (see How the pipeline skips finished work). `BidsDcmToNii` skips a subject whose `raw` folder is not empty. When new subjects arrive, set `ProcessSubjects` to the new subjects only.

The steps below use the same two options, where a step has them:

- `ProcessSubjects`: `All` for every subject, or a list of subject names.
- `VersionCheck`: `False` (default) skips finished work. `True` also redoes work made with an older version of QMRITools.

You can also run the steps from a window, with `MuscleBidsGUI[dir]` (see MuscleBidsGUI).

1. BidsDcmToNii

Why: Your scanner data is in DICOM format. The later steps need NIfTI files, which this step makes.

Goes in: the DICOM folders in `01_sourcedata`.

Comes out: raw NIfTI files in `02_rawdata/sub-<subject>/ses-<session>/raw`, and a log file for the run.

```
BidsDcmToNii[dir, ProcessSubjects -> All]
```

This step has no `VersionCheck` option. If a `raw` folder already has files, this step skips that subject. To convert a subject again, empty its `raw` folder first.

2. MuscleBidsConvert

Why: The raw files do not yet say what they contain. This step uses the `Label` (protocol name) of each data type in your config to sort the files into the data types, such as `dix` (Dixon) and `dwi` (diffusion), and gives them BIDS names.

Goes in: the raw NIfTI files in `raw`.

Comes out: BIDS-named NIfTI and JSON files in the type folders next to `raw`, such as `dix` and `dwi`. By default, the raw files are deleted after conversion.

```
MuscleBidsConvert[dir, ProcessSubjects -> All]
```

This step has no `VersionCheck` option.

Tip: the `Label` in your config must match the protocol name on the scanner exactly. After `BidsDcmToNii`, run `ViewProtocolNames[dir]` to see the protocol names per subject, and `CheckConfigLabels[dir]` to see which of your labels are missing (see Checking and viewing your study).

3. MuscleBidsProcess

Why: This step fits the physical models to your images. For example, it calculates fat fractions from Dixon data, T2 values from multi-echo data, and diffusion tensors from DTI data. These maps are the numbers you analyse later.

Goes in: the BIDS-named files in `02_rawdata`.

Comes out: the parameter maps in `03_derivatives`, in the native space of the scan.

```
MuscleBidsProcess[dir, ProcessSubjects -> All, VersionCheck -> False]
```

4. MuscleBidsMerge

Why: A muscle group often does not fit in one field of view, so it is scanned in several parts. This step joins the parts. It then moves all data types into one common space, so that the same muscle lines up across all images.

Goes in: the parameter maps in `03_derivatives`.

Comes out: the joined and registered maps in `04_merged`. Only data types that have a `Merging` section in the config are merged.

```
MuscleBidsMerge[dir, ProcessSubjects -> All, VersionCheck -> False]
```

5. MuscleBidsSegment

Why: Analysis needs to know which pixels belong to which muscle. This step draws the muscle outlines automatically with a neural network. Without segmentations, you cannot run the analysis. If you already have manual segmentations, you can skip this step and put your files in the `seg` folder instead.

Goes in: the merged data in `04_merged`. Only data types that have a `Segment` section in the config are segmented.

Comes out: the segmentation files in the `seg` folder of each subject and session in `04_merged`.

```
MuscleBidsSegment[dir, ProcessSubjects -> All, VersionCheck -> False]
```

6. MuscleBidsTractography

Why: Diffusion data shows the direction of the muscle fibres. This step follows the fibres through the muscle (tractography) and makes maps of them. If you have segmentations, it also splits the fibres per muscle. Without a segmentation, you get tracts for the whole volume only.

Goes in: the diffusion tensors in `04_merged`, and the segmentations if you have them. Only data types that have a `Tractography` section in the config are tracked.

Comes out: in the `dwi` folder in `04_merged`, a tract file (`.trk`) for the whole volume. With a segmentation, also the tracts per muscle and maps of the tracts (density, length, angle, seed density and curvature).

```
MuscleBidsTractography[dir, ProcessSubjects -> All, VersionCheck -> False]
```

7. MuscleBidsAnalysis

Why: This is the final result. For each subject, it measures the volume of every muscle and summary values (usually the median) of every muscle in the parameter maps you choose in the config. It also makes images of your data. The results are written to Excel, so you can open them in any spreadsheet program.

Goes in: the merged data and segmentations in `04_merged`, and the analysis settings in your config file.

Comes out: one Excel file per subject and session, and images, in `05_analysis`. All subjects are also combined into one file, named `All_<date>.xlsx`, in the same folder.

```
MuscleBidsAnalysis[dir, ProcessSubjects -> All, VersionCheck -> False]
```

Rarely needed settings for this part: see Pipeline step options in the appendix.

A complete run, step by step

This is the recommended order, from a study folder with dicom data to the analysis sheets. Each step says why it exists and what to check before and after it. Paste each numbered step into its own cell of a notebook saved in the study folder and run it with Shift+Enter. Go on to the next cell only when its “After” check is fine. Lines between `(* ... *)` are comments that Mathematica ignores. Do not skip the checks: most problems are a wrong name in the config, and the checks show them early.

```
(* Load QMRITools *)
Needs["QMRITools`"]

(* dir is the study folder: the folder that holds this notebook and config.json *)
dir = NotebookDirectory[];
subs = All;   (* All until you pick the subjects in step 2 *)

(* 1. Check the config file *)
(* Why: a wrong folder name or Label is ignored without an error, so read the config first. *)
(* Before: config.json is in dir and is valid JSON (no trailing commas). *)
(* After: every dataset shows the Label, Type and Method you expect. *)
ViewConfig[dir]

(* 2. Pick the subjects *)
(* Why: you process only the subjects you choose, and you can add new ones later. *)
(* Before: the dicom folders are in 01_sourcedata, named subject_session, for example 001_1. *)
(* After: click "Copy selected list to clipboard", then paste it in place of All, for example subs = {"001_1"}. *)
SelectSubjects[dir]

(* 3. Convert the dicom files to nifti *)
(* Why: dcm2niix makes the .nii files that all later steps read. *)
(* Before: the raw folder of a subject is empty. A subject with files in raw is skipped. *)
(* After: 02_rawdata/sub-x/ses-y/raw has .nii and .json files, and the DcmToNii log in 02_rawdata says "Folder was converted". *)
BidsDcmToNii[dir, ProcessSubjects -> subs]

(* 4. List the protocol names of the scanner *)
(* Why: the Label values in the config must match these names exactly. *)
(* Before: step 3 is done. *)
(* After: no duplicate names in red. Every Label you need is in the list. *)
ViewProtocolNames[dir, ProcessSubjects -> subs]

(* 5. Check that every Label is present *)
(* Why: a Label that is not in the scans is never converted, and nothing tells you so. *)
(* Before: step 4 is done. *)
(* After: all rows say yes. For a missing row, fix Label in config.json and click Reload. *)
CheckConfigLabels[dir, subs]

(* 6. Convert to BIDS names *)
(* Why: this gives the files the names that the rest of the pipeline looks for. *)
(* Before: step 5 has no missing Labels. *)
(* After: step 7 has no missing rows. *)
(* DeleteAfterConversion -> False keeps the raw files while you learn. Remove it to delete them after conversion. *)
MuscleBidsConvert[dir, ProcessSubjects -> subs, DeleteAfterConversion -> False]

(* 7. Check the converted files *)
(* Why: finds a failed conversion now, before the slow processing steps. *)
(* Before: step 6 is done. *)
(* After: every row says yes. For a missing row, check the Label (step 5) and run step 6 again. *)
CheckBidsOutput[dir, "Convert", ProcessSubjects -> subs]

(* 8. Fit the quantitative maps *)
(* Why: makes the fat fraction, T2* and similar maps for each stack. *)
(* Before: step 7 has no missing rows. *)
(* After: 03_derivatives has .nii files, and the Process tab of ManageCheckFiles shows done. *)
(* Add VersionCheck -> True to this and the next steps to redo results made with an older QMRITools version. *)
MuscleBidsProcess[dir, ProcessSubjects -> subs]

(* 9. Merge the stacks *)
(* Why: joins the stacks of one dataset into one volume and puts it in the space of the Target. *)
(* Before: step 8 is done. A dataset is only merged if it has a Merging block. *)
(* After: 04_merged has the merged .nii files. *)
MuscleBidsMerge[dir, ProcessSubjects -> subs]

(* 10. Segment the muscles *)
(* Why: gives the muscle masks that the analysis uses. *)
```

```
(* Before: step 9 is done. A dataset is only segmented if it has a Segment block. *)
(* After: 04_merged/.../seg has seg_auto .nii files. A check status of error means an input was missing. *)
MuscleBidsSegment[dir, ProcessSubjects -> subs]

(* 11. Tractography (only for datasets with a Tractography block, for example DTI) *)
(* Why: follows the fibres of DTI data and splits them per muscle. *)
(* Before: steps 9 and 10 are done. *)
(* After: .trk files in 04_merged/.../dwi. Datasets without a Tractography block are skipped. *)
MuscleBidsTractography[dir, ProcessSubjects -> subs]

(* 12. Analysis: make the Excel sheets *)
(* Why: writes one table per subject and one table for all subjects. *)
(* Before: step 10 is done, because the analysis uses the segmentation. Tract based types also need step 11. *)
(* After: 05_analysis/sub-x/ses-y has .xlsx files, and 05_analysis has All_<date>.xlsx. Open one and check the values. *)
(* "None" writes only the sheets. Use "All" after you add an Images block to the config. *)
MuscleBidsAnalysis[dir, ProcessSubjects -> subs, BidsOutputImages -> "None"]

(* 13. Overview of all checks *)
(* Why: one window with protocol names, Labels, conversion output and check files. *)
(* After: nothing red or missing. Use the same BidsOutputImages value as in step 12. *)
MuscleBidsCheck[dir, ProcessSubjects -> subs, BidsOutputImages -> "None"]

(* Rerun after new subjects arrive *)
(* Why: finished steps are skipped, so you can run the chain again. *)
(* Set subs to the new names only, then run steps 3 to 12 again. Step 6 with DeleteAfterConversion -> False would convert old
subjects again. *)

(* Rerun after you change a setting in config.json *)
(* Why: a finished step keeps its check file, so a changed setting is not used until you delete that file. *)
(* Delete the check files of the changed step and of all later steps, then run those steps again. *)
ManageCheckFiles[dir, ProcessSubjects -> subs, BidsOutputImages -> "None"]
```

Folder layout after a run

With the minimal config and one subject `001_1`, the study folder looks like this. Files are removed from `raw` after conversion unless you use `DeleteAfterConversion -> False`.

```
<study>/
  config.json
  01_sourcedata/                    dicom folders, copied from the scanner
    001_1/
  02_rawdata/
    DcmToNii_<date>.log             one log per BidsDcmToNii run
    sub-001/ses-1/
      raw/                          dcm2niix output
      dix/                          converted files: sub-001_ses-1_stk-DIXON1_megre_dix_*.nii and .json
      sub-001_ses-1_BIDSConvert.log
  03_derivatives/sub-001/ses-1/dix/ fitted maps: sub-001_ses-1_stk-DIXON1_megre_dix_wat.nii, *_check.json
  04_merged/sub-001/ses-1/
    dix/                            merged maps: sub-001_ses-1_megre_dix_wat.nii
    seg/                            segmentation: sub-001_ses-1_seg_auto_megre_dix_outph.nii
  05_analysis/
    sub-001/ses-1/                  sub-001_ses-1.xlsx and .wxf
    All_<date>.xlsx                 all subjects in one table
```

Most steps write `*_check.json` files, which is how the next run knows what is done. `ManageCheckFiles` (the last line of the block) lists them.

Starting from an example config

The folder `BIDS Example`, where this manual is also stored, has five configs. Each one shows a different part of the pipeline. Copy the one closest to your study, then change the values for your scans.

Config	Fits a study with
`config.json`	The basic study: Dixon, diffusion with tractography, T2 mapping and a TSE Dixon, with one analysis. Start here.
`motion_config.json`	Six stacks per scan that are joined, with Dixon, diffusion and T2 mapping.

Config	Fits a study with
`motor_config.json`	Several body regions (head, calf, body, shoulder) with the same scans. No `analysis` section yet.
`twitch_config.json`	Dixon data reconstructed by the scanner, and segmentation in 2D slices. No `analysis` section yet.
`boch_config.json`	Two analyses next to each other (`OS` and `US`). Its dataset names contain `_`, which you should avoid in your own config.

Minimal config

This is the basic config with one dataset, the `Dixon` dataset of `config.json`, and the analysis for it. Copy it into `config.json` in the study folder. The values you must change are listed below the block.

```
{
    "folders": {
        "dicomData": "01_sourcedata",
        "rawData": "02_rawdata",
        "derivedData": "03_derivatives",
        "mergeData": "04_merged",
        "analysis": "05_analysis"
    },
    "datasets": {
        "Dixon": {
            "Label": ["DIXON-1", "DIXON-2", "DIXON-3", "DIXON-4", "DIXON-5", "DIXON-6"],
            "Class": "Stacks",
            "Type": "megre",
            "Suffix": "dix",
            "Process": {
                "Method": "Dixon"
            },
            "Merging": {
                "Overlap": 5,
                "Target": ["megre", "dix", "outph"],
                "Moving": "wat",
                "Process": ["wat", "fat", "inph", "outph", "watfr", "fatfr", "t2star"]
            },
            "Segment": {
                "Target": ["megre", "dix", "outph"],
                "Location": "Body"
            }
        }
    },
    "analysis": {
        "Segmentation": {
            "Type": ["seg", "auto", "megre", "dix", "outph"],
            "Labels": [799, "Body"]
        },
        "Analysis": {
            "Types": [
                ["megre", "dix", ["watfr", "fatfr", "t2star"]]
            ]
        }
    }
}
```

Change these values for your study:

Key	Change it to	Why
`datasets.Dixon.Label`	The exact protocol names from step 4. Keep one name per stack.	The pipeline finds each scan by its protocol name.
`datasets.Dixon.Class`	`"Stacks"` for a list of several Labels. Use `"Volume"` with one Label written as text.	Tells the pipeline how the Labels are combined.

Key	Change it to	Why
`datasets.Dixon.Type` and `Process.Method`	`"megre"` or `"tse"` with `"Dixon"`, `"Dixon-B"`, `"Dixon-P"`, `"Dixon-S"` or `"Dixon-A"`; `"dwi"` with `"DTI"`; `"mese"` with `"EPGT2"` or `"Exp"`.	Selects the reconstruction. If you change `Type` or `Suffix`, change the lists that use them too (`Merging.Target`, `Segment.Target`, `analysis.Segmentation.Type`, `analysis.Analysis.Types`).
`datasets.Dixon.Segment.Location`	Keep `Body` (the default) for the full-body muscle model.	Selects the muscle segmentation model. Older region sets are listed in the appendix.
`analysis.Segmentation.Labels`	`[799, "Body"]` for the default `Body` location.	The number is the highest muscle label (bones are 800 and up and are not analysed). The word selects the muscle names. See Segmentation.

Also check `Merging.Overlap`. It is the number of slices that neighbouring stacks share. Use `0` if your stacks do not overlap. Then the stacks are joined without motion correction between them.

The config file

The config file tells QMRITools what data to expect, what to process, and where to put the results. It is a plain text file in JSON format, always named `config.json`, and it lives in your study folder. Each study has one main config file. You can open it in a readable view with `ViewConfig[dir]`, where `dir` is your study folder. `ViewConfig["default"]` shows all default values. They are also listed in Default config in the appendix.

You can also place a `config.json` file in a single subject folder. It changes the datasets for that subject only. See Per-subject settings for details.

Writing the file (JSON)

JSON is a simple text format. The rules are few:

- Settings are written as `"name": value`, separated by commas, with **no comma after the last one**.
- `{ ... }` groups settings, `[ ... ]` is a list. Text is in double quotes (`"Dixon"`), numbers and `true` / `false` are not.
- Comments are not allowed.

`ViewConfig[dir]` shows the file as tables, so you can see whether it was read correctly.

The four sections

The config file has four sections. You only write the sections you need. The others get default values.

Section	What it is for
`folders`	The names of the folders that QMRITools creates and uses during processing.
`conversion`	How DICOM files are converted to NIfTI files.
`datasets`	The data you want to process and how each type of data is handled.
`analysis`	Which results are analyzed and which images are made.

The processing steps write their results to separate folders, one folder per step. This way you can always see what each step produced, and you can check a step before you go on to the next one.

What happens when a key is missing

If you leave out a key, QMRITools uses a default value for it. This means you can start with a config file that only contains the keys you want to change. For `folders`, each folder name has its own default. For `conversion`, the default applies when you leave out the whole section.

Two sections have no default. You must write them when you use the steps that need them:

- `datasets`, which lists the data to process. Every step except the DICOM to NIfTI conversion needs it.
- `analysis`, which is only needed for the analysis step.

folders

The `folders` section sets the names of the five folders. You do not have to write this section. If you leave out a folder, its default name is used.

Key	What it is for	Default	Used by
`dicomData`	Holds the DICOM files that come from the scanner. You put them here before you start.	`01_sourcedata`	`BidsDcmToNii`
`rawData`	Holds the converted NIfTI files. After conversion, this folder contains your BIDS dataset.	`02_rawdata`	`BidsDcmToNii`, `MuscleBidsConvert`, `MuscleBidsProcess`

Key	What it is for	Default	Used by
`derivedData`	Holds the processed data, still in the space of each scan.	`03_derivatives`	`MuscleBidsProcess`, `MuscleBidsMerge`
`mergeData`	Holds the merged data, all in one common space. Segmentation and tractography also read and write here.	`04_merged`	`MuscleBidsMerge`, `MuscleBidsSegment`, `MuscleBidsTractography`, `MuscleBidsAnalysis`
`analysis`	Holds the results of the analysis: `.xlsx` sheets, `.wxf` files and images.	`05_analysis`	`MuscleBidsAnalysis`

Example, with the default names:

```
"folders": {
  "dicomData": "01_sourcedata",
  "rawData": "02_rawdata",
  "derivedData": "03_derivatives",
  "mergeData": "04_merged",
  "analysis": "05_analysis"
}
```

Example, with a different name for the raw data folder only:

```
"folders": {
  "rawData": "02_nifti"
}
```

All other folders keep their default names.

conversion

The `conversion` section chooses the version of dcm2niix that converts your DICOM files to NIfTI. dcm2niix is the free converter that QMRITools uses. Leave this section out unless the conversion fails.

Key	What it is for	Default
`Version`	Which dcm2niix version to use. See the list below.	`"Own"` on Windows, `1` on macOS

Example:

```
"conversion": {
  "Version": 1
}
```

Write the value `1` as a number and not as text (`"1"`). The text `"1"` is not a known version, so the conversion fails.

If you write a `conversion` section, include `Version`.

The values you can use depend on your system:

- On Windows: `"Own"` (the default) is the dcm2niix build that comes with QMRITools. If the conversion fails, try `1`, another build that comes with QMRITools. Older builds (`17`, `19`, `20`, `21`, `23`, `24`, `25`) are also available for data that needs them.
- On macOS: only `1` is available.

The `datasets` and `analysis` sections are explained in Datasets and Analysis.

Datasets: describing your scans

The `datasets` section tells the pipeline which scans to use and what to do with them. Each entry describes one acquisition (or a group of scans that belong together). The name you give the entry is its **dataset name**. The pipeline uses it to refer to that dataset in later steps.

Every dataset needs a description (what the scan is and where it lives). Later steps add their own blocks to the same entry: `Process` (turn the raw scan into maps), `Merging` (combine several pieces), `Segment` (draw muscle masks) and `Tractography` (fibre tracking). A step skips a dataset that has no block for it.

A first dataset

```
"datasets": {
  "DIXOS": {
    "Label": ["DIXON-1", "DIXON-2", "DIXON-3"],
    "Class": "Stacks",
    "Type": "megre",
    "Suffix": "dix",
    "Process": {
      "Method": "Dixon"
    }
  }
}
```

This reads three Dixon scans with the protocol names `DIXON-1`, `DIXON-2` and `DIXON-3`, and processes them with the `Dixon` method.

Data description keys

Key	What it is for	Default / allowed values
`Label`	The protocol name(s) of the scan(s), as they appear in the converted `.json` files. After the DICOM conversion, `ViewProtocolNames` shows the names that exist and `CheckConfigLabels` checks that your labels are present. A leading `WIP` is ignored.	A text (one scan) or a list of text (several scans). Required.
`Class`	How the scans in `Label` belong together. See the next section.	`"Volume"` (default). Also `"Volumes"`, `"Stacks"`, `"Chunks"`, `"Repetitions"`, `"Acquisitions"`.
`Type`	The kind of scan. It decides the folder the data is stored in and which methods you can use.	See the table below. Required.
`Suffix`	A short word that is added to the file name, so you can recognise the data.	Empty text by default. Use `"dix"` for Dixon data, `"t2"` for T2 mapping, `"dti"` for diffusion.
`Process`	How the raw scan is turned into maps. Its key `Method` is required.	See the method table below. Optional.
`Merging`	How this dataset is combined with others. Required if you want the merge step.	See Merging.
`Segment`, `Tractography`	Muscle masks and fibre tracking on this dataset.	See Segment and Tractography. Optional.

Class: how repeated scans are grouped

`Class` tells the pipeline how to treat the scans listed in `Label`. Think of it as "what are these scans to each other?" The class also gives the word that is put in front of the label in the file names (shown in brackets).

- **Volume** (default, `vol`): one scan, one protocol name. The `Label` is a single text, for example `"Label": "T1map"`.
- **Volumes** (`vol`): several diffusion scans that are joined into one dataset, for example a diffusion scan split into parts. The first label is used for the file name.
- **Stacks** (`stk`): several stacks (slabs) of the same body part, taken at different positions. They are merged later, using an overlap between neighbouring stacks.
- **Chunks** (`chunk`): like Stacks, but the file names use the word "chunk" instead of "stk".
- **Repetitions** (`rep`): the same scan done several times. Each label is one repetition.
- **Acquisitions** (`acq`): several separate acquisitions. Each label is kept as its own acquisition.

Example: three Dixon slabs taken at different positions:

```
"Label": ["DIXON-1", "DIXON-2", "DIXON-3"],
"Class": "Stacks"
```

A single T1 map:

```
"Label": "T1map",
"Class": "Volume"
```

For `Volume` you must give a single text. For every other class give a list, even with one item. If the combination is wrong, the dataset is skipped and the log reports a wrong data description.

Type and method

`Type`	Stored in the folder	Converted and processed	`Process.Method` values
`megre`, `tse`	`dix`	yes	`Dixon`, `Dixon-S`, `Dixon-P`, `Dixon-B`, `Dixon-A`
`dwi`	`dwi`	yes	`DTI`
`mese`	`quant`	yes	`EPGT2`, `Exp`
`T1`, `T2`, `wT2`	`quant`	no	
`T1w`, `T1w-FS`, `T2w`, `T2w-FS`	`anat`	no	

Use `megre` (or `tse`) for Dixon data. The types marked "no" are known to the pipeline for the folder name, but the pipeline does not convert or process them. Any other `Type` gives a warning and the folder `miss`.

Dataset names and the `Key`

The dataset name is the text you give the entry, for example `"DIXOS"`. Use only letters and digits, for example `DIXOS` or `DIXUS`. Other characters can make the later steps fail.

The pipeline uses the dataset name as the `Key` in file names.

The `Key` is only used in file names when two or more datasets have the same `Type` and `Suffix`. Then:

- every dataset gets the key in the names of its merged files (for example `chunk-DIXOS` or `stk-DIXOS`), and
- every `Target` and `Segmentation` list should start with the key of the dataset you mean, for example `["DIXOS", "megre", "dix", "wat"]`.

If all `Type` and `Suffix` combinations are different, the file names have no key and you write `["megre", "dix", "wat"]`.

There is one exception: two datasets that store their data in the same folder must still be told apart. `megre` and `tse` datasets both use the folder `dix`, so a target for the `tse` dataset starts with its dataset name, for example `["TSE", "tse", "dix", "outph"]`. The file names still have no key.

Example of two datasets that need keys, because both are `megre` with the suffix `dix`:

```
"datasets": {
  "DIXOS": {
    "Label": ["DIXON-OS-1", "DIXON-OS-2"],
    "Class": "Stacks",
    "Type": "megre",
    "Suffix": "dix"
  },
  "DIXUS": {
    "Label": ["DIXON-US-1", "DIXON-US-2"],
    "Class": "Stacks",
    "Type": "megre",
    "Suffix": "dix"
  }
}
```

How the output files are named

The pipeline builds the file names from the data description. You only need to know the parts you can see in the output:

- The class word and the label are written at the front, for example `stk-DIXON1` for a Stacks label `DIXON-1` (the dash and underscore are removed from the label).
- The `Type` follows (`megre`, `dwi`, `mese`), then the `Suffix` (`dix`, `dti`, `t2`).
- The last part names the image, for example `real` for a raw Dixon image and `wat` for the water image after processing.

Example, one scan from a dataset with `Class` `"Stacks"` and `Label` `"DIXON-1"`, `Type` `megre` and `Suffix` `dix`:

```
sub-1_ses-1_stk-DIXON1_megre_dix_real.nii
```

After merging, the label is replaced by the key, and only when there are duplicates, as described above:

```
sub-1_ses-1_stk-DIXOS_megre_dix_wat.nii
```

Without duplicates the merged name has no key: `sub-1_ses-1_megre_dix_wat.nii`. The class word (`vol`, `stk`, `chunk`, `rep`, `acq`) is the one listed under Class.

Rarely needed settings for this part: see Dataset options in the appendix.

Processing, merging, segmentation and tractography

Each dataset in `datasets` can have one block for every pipeline step after the conversion: `Process`, `Merging`, `Segment` and `Tractography`. A step only works on datasets that have its block, so you write just the blocks you need. Keys that most studies never change are listed in an *Advanced* table at the end of each block.

Process

Why: the `Process` block tells the pipeline how to turn the converted scanner data of one dataset into useful maps, such as fat fraction, T2, or fractional anisotropy. Choose the `Method` that matches your acquisition. Without a `Method` no maps are calculated for the dataset (diffusion data is still denoised and registered), so nothing useful reaches the later steps (merging, segmentation, tractography, analysis).

Defaults: there is no default `Method`. You must set it. Every other key has a default that is listed in the tables below.

Methods:

`Method`	Use it for	Output (examples of the suffixes you get)
`Dixon`	Multi-echo gradient echo (Dixon) scans (`Type` `megre` or `tse`) stored as magnitude and phase, or real and imaginary, images per echo. QMRITools calculates the water and fat maps itself.	`wat`, `fat`, `watfr`, `fatfr`, `t2star`, `r2star`, `inph`, `outph`, `snr`
`Dixon-B`	Like `Dixon`, but for data stored as one 4D file with all echoes. You give the echo times with `EchoTime`.	same as `Dixon`
`Dixon-P`	Dixon data that the scanner already reconstructed, as one 4D file or as separate files.	the maps listed in `Types`
`Dixon-S`	Dixon data that the scanner already reconstructed, as separate series.	the maps listed in `Types`
`Dixon-A`	Dixon data where you do not know (or it differs per scan) if the scanner reconstructed it. The type is detected automatically.	the scanner maps, or the QMRITools maps when only the raw echoes are there
`DTI`	Diffusion-weighted images (`Type` `dwi`). Fits a diffusion tensor.	`s0`, `md`, `fa`, `rd`, `l1`, `l2`, `l3`, `tens`, `snr0`
`EPGT2`	Multi-echo spin echo T2 mapping (`Type` `mese`). Needs the RF pulse settings. The fat T2 is calibrated on the subcutaneous fat, which is assumed to be 90% fat and 10% water. `fatfr` is the lipid fraction, the same quantity Dixon gives.	`t2w`, `t2f`, `wat`, `fat`, `fatfr`, `b1`
`Exp`	Simple T2 mapping (`Type` `mese`) with a single exponential fit.	`t2`, `s0`

The full list of suffixes is at https://www.qmritools.com/bids/ents_suffs/.

Keys:

Key	What it is for	Default / allowed values
`Method`	Chooses the processing.	see the table above. Required
`Types`	For `Dixon-P`, `Dixon-S` and `Dixon-A`. Lists the scanner maps you want. Use a plain list when all maps are in one 4D file (`Dixon-P`), in the order of the volumes. Use a list of pairs `[map name, tag]` when each map is in its own file (`Dixon-P`, `Dixon-S`). The tag is the text that marks the map in the file name (`Dixon-P`) or in the series name (`Dixon-S`).	Required for `Dixon-P` and `Dixon-S`. Leave it out for `Dixon-A` (it uses `["wat","fat","inph","outph"]`). Example `["wat","fat","inph","outph"]`
`RicianCorrection`	For `DTI`. Magnitude images have a noise bias that raises low signals (high b-values), which is corrected after the denoising. Set it to `false` to skip this.	`true`
`GradientCorrection`	For `DTI`. Corrects the diffusion for the non-linear gradients of the scanner. Set it to the coil name. The scan needs an offset in its json file (written when the data is converted), otherwise no correction is done.	`false` (no correction). Example `"WA1"`
`IVIMCorrection`	For `DTI`. Removes the fast perfusion part of the signal before the tensor is fitted.	`true`
`Settings`	For `EPGT2`. The two RF pulses (excitation and refocusing) of your scan, each as `[name, flip angle in degrees, [gradient strength, duration, bandwidth]]`. Copy the values from your sequence protocol. You can also give just the two flip angles, `[90, 180]`; then the slice profile and `Shift` are not used.	Required for `EPGT2`
`Shift`	For `EPGT2` with pulse settings. Set to `true` to correct the fat signal for the chemical shift. The field strength is read from the scan (`MagneticFieldStrength` in the JSON file), 3 T if it is missing.	`false`

Example (Dixon):

```
{
  "datasets": {
    "Dixon": {
      "Label": ["DIXON-1", "DIXON-2", "DIXON-3"],
      "Class": "Stacks",
      "Type": "megre",
      "Suffix": "dix",
      "Process": {
        "Method": "Dixon"
      }
    }
  }
}
```

Example (scanner reconstructed Dixon in one 4D file):

```
{
  "datasets": {
    "TSE": {
      "Label": ["TSE-1", "TSE-2", "TSE-3"],
      "Class": "Stacks",
      "Type": "tse",
      "Suffix": "dix",
      "Process": {
        "Method": "Dixon-P",
        "Types": ["wat", "inph", "outph", "fat"]
      }
    }
  }
}
```

Example (diffusion):

```
{
  "datasets": {
    "DTI": {
      "Label": ["DTI-1", "DTI-2", "DTI-3"],
      "Class": "Stacks",
      "Type": "dwi",
      "Suffix": "dti",
      "Process": {
        "Method": "DTI",
        "GradientCorrection": "WA1"
      }
    }
  }
}
```

Example (T2 mapping):

```
{
  "datasets": {
    "EPGT2": {
      "Label": ["T2-1", "T2-2", "T2-3"],
      "Class": "Stacks",
      "Type": "mese",
      "Suffix": "t2",
      "Process": {
        "Method": "EPGT2",
        "Settings": [
          ["sg_150_100_167", 90, [1.5991, 5.661, 538]],
          ["sinc_centre", 180, [1.3326, 2.9376, 340]]
        ],
        "Shift": true
      }
    }
  }
}
```

The `Settings` values in this example are the pulse settings of the example protocol. Use the values of your own scanner.

Rarely needed settings for this part: see Process options in the appendix.

Merging

Why: merging brings all the maps of one dataset into one common space, and joins the chunks of a scan into one volume. You need this so that maps from different scans can be compared, segmented and tracked together. The `Target` decides the space: every map is moved to the space (and resolution) of the target. A target is written as `[type, suffix, map]` of the image whose space you want. It can belong to another dataset (for example diffusion data moved to the Dixon space); that dataset must then be processed and merged first. A dataset without a `Merging` block is skipped in this step.

Defaults: there is no default `Target`, `Moving` or `Process`. You must set them. `Overlap` defaults to `0`.

Keys:

Key	What it is for	Default / allowed values
`Target`	The map that defines the common space, written like a file name: type, suffix and map name, for example `["megre","dix","outph"]`.	none. Required
`Moving`	The map of this dataset that is registered (aligned) to the target. Use one map name, for example `"wat"`. It must also be in `Process`.	none. Required
`Process`	The maps that you want to keep after merging. Only these maps are used in the later steps. Use the suffixes that the processing created, for example `["fat","watfr","snr"]`.	none. Required
`Overlap`	Number of slices that two chunks overlap. Set this to the real overlap of the acquisition. When it is above `0`, the overlapping slices are blended into one volume and the chunks are first aligned to each other. When it is `0`, the chunks are simply put after each other.	`0`. Also `[overTarget, overNative]`

Example:

```
{
  "datasets": {
    "Dixon": {
      "Label": ["DIXON-1", "DIXON-2", "DIXON-3"],
      "Class": "Stacks",
      "Type": "megre",
      "Suffix": "dix",
      "Process": {
        "Method": "Dixon"
      },
      "Merging": {
        "Overlap": 5,
        "Target": ["megre", "dix", "outph"],
        "Moving": "wat",
        "Process": ["t2star", "r2star", "inph", "outph", "wat", "fat", "watfr", "fatfr", "snr"]
      }
    }
  }
}
```

If the `Target` comes from another dataset, all maps are moved to the space of that target. The diffusion tensor (`tens` , and `fasc`) is the exception: it is aligned too, but keeps its own voxel size, so that its directions stay correct. A list `[overTarget, overNative]` for `Overlap` gives the overlap in slices for the target resolution and for these maps.

If the `Target` is the same map as `Moving` (the same dataset), the chunks are aligned to each other, with the first chunk as the reference. Set `Motion` to `false` to skip this.

If two datasets have the same `Type` and `Suffix` , every `Target` must start with the name of the dataset that it comes from, for example `["Dixon","megre","dix","outph"]` , and the merged file names then contain that dataset name. Use dataset names with only letters and digits for this.

Rarely needed settings for this part: see Merging options in the appendix.

Segment

Why: segmentation draws the outline of each muscle automatically. You need it for the per-muscle analysis and for tracking per muscle. Without a `Segment` block the dataset is not segmented. You can then add your own segmentation instead (see below).

Defaults: `Location` `"Body"` , `Dimensions` `"3D"` , `Device` `"CPU"` . `Target` has no default and you must set it.

Keys:

Key	What it is for	Default / allowed values
`Target`	The map the muscles are found on, written like a file name. The example uses the out-of-phase Dixon image. Give a list of lists to make one segmentation per map. It must be a map you kept in `Merging.Process` .	none. Required. Example `["megre","dix","outph"]`
`Location`	The part of the body to segment. Keep the default `"Body"` , the full-body muscle model.	`"Body"` . The other values are for rare cases, see Segment options.

Example:

```
{
  "datasets": {
    "Dixon": {
      "Label": ["DIXON-1", "DIXON-2", "DIXON-3"],
      "Class": "Stacks",
      "Type": "megre",
      "Suffix": "dix",
      "Process": {
        "Method": "Dixon"
      },
      "Merging": {
        "Overlap": 5,
        "Target": ["megre", "dix", "outph"],
        "Moving": "wat",
        "Process": ["wat", "watfr", "fatfr", "outph"]
      },
      "Segment": {
        "Target": ["megre", "dix", "outph"],
        "Location": "Body"
      }
    }
  }
}
```

The segmentation is made with a neural network and is saved with the name `seg_auto_<target>`, for example `seg_auto_megre_dix_outph`.

Your own segmentation: leave out the `Segment` block and add your own file:

- A NIfTI file in the `seg` folder of the subject and session in the merged data (for example `04_merged/sub-001/ses-1/seg/`), named like the automatic one but with `man` instead of `auto`: `sub-001_ses-1_seg_man_megre_dix_outph.nii`.
- One whole-number label for each muscle (and each side), on the same voxel grid as the `Target` map of that subject.
- The label numbers must fit the labels you give to the analysis (`Segmentation.Labels`, for example `[799, "Body"]`), or you give the path to your own label file.

Then use `["seg","man","megre","dix","outph"]` where a later step asks for a segmentation.

Rarely needed settings for this part: see Segment options in the appendix.

Tractography

Why: tractography follows the direction of the muscle fibres through the diffusion tensor and draws them as lines (tracts). The tracts can be split per muscle when you give a segmentation. You need this step only for `DTI` datasets.

Defaults: there are no defaults for `Target` and `Stopping`. You must set both. `Segmentation` is optional. The maps you use here must be listed in `Merging.Process` of the dataset, because tractography works on the merged data.

Keys:

Key	What it is for	Default / allowed values
`Target`	The tensor map to track on, normally the `tens` map of your `DTI` dataset.	none. Required. Example `["dwi","dti","tens"]`
`Stopping`	The rules that stop a tract. Each rule is a map and the allowed range `[min, max]`. A tract stops as soon as one rule is broken. Use this to keep the tracts inside the muscle.	none. Required. Example `[[["dwi","dti","fa"],[0.05,0.65]], [["dwi","dti","md"],[0.5,2.5]]]`
`Segmentation`	The segmentation used to split the tracts per muscle. Leave it out to track the whole volume without splitting.	not set. Example `["seg","auto","megre","dix","outph"]`

Example:

```
{
  "datasets": {
    "DTI": {
      "Label": ["DTI-1", "DTI-2", "DTI-3"],
      "Class": "Stacks",
      "Type": "dwi",
      "Suffix": "dti",
      "Process": {
        "Method": "DTI"
      },
      "Merging": {
        "Overlap": 5,
        "Target": ["megre", "dix", "wat"],
        "Moving": "s0",
        "Process": ["s0", "snr0", "fa", "md", "rd", "tens"]
      },
      "Tractography": {
        "Target": ["dwi", "dti", "tens"],
        "Segmentation": ["seg", "auto", "megre", "dix", "outph"],
        "Stopping": [
          [["dwi", "dti", "fa"], [0.05, 0.65]],
          [["dwi", "dti", "md"], [0.5, 2.5]]
        ]
      }
    }
  }
}
```

The `Segmentation` in this example is the one made for the Dixon dataset (see Segment). If `Segmentation` is not set, the whole volume is tracked, but the tracts are not split per muscle.

Rarely needed settings for this part: see Tractography options in the appendix.

Per-subject settings

Why: sometimes one subject or session needs different settings, for example a different overlap or a different segmentation location. You can change these for one subject without changing the main config for all subjects.

How: put a file named `config.json` in the folder of that subject in the source data folder, for example `01_sourcedata/001_1/config.json`. Its content is a patch on the `datasets` section of the main config. Use the dataset names as the top-level keys, and only write the keys you want to change. You do not need to copy the full config.

```
{
  "DTI": {
    "Merging": {
      "Overlap": 3
    },
    "Tractography": {
      "Stopping": [
        [["dwi", "dti", "fa"], [0.05, 0.6]]
      ]
    }
  }
}
```

The settings are merged with the main config for this subject only. The keys in your file replace the main settings; a list (such as `Stopping` or `Label`) is replaced as a whole. Other keys of the same dataset keep their main values.

The file is copied with the data to the later folders as `sub-<subject>_ses-<session>_config.json`, so it applies to all the steps from the conversion to the tractography. The analysis step does not use per-subject files. The log shows `Using custom config` when a subject has one.

To check for each subject and session that the `Label`s of the datasets are present in the scan (including the per-subject settings), use `CheckConfigLabels[dir, subs]`, where `dir` is your study folder and `subs` is `All` or a list of subjects.

To see which subjects have a per-subject config, and what each one changes, use `CheckCustomConfigs` or the **Custom config** tab of `MuscleBidsConfig` (see CheckCustomConfigs). It shows one row for each changed setting, with the value of the subject next to the value of the study. It also finds custom configs that were not carried through the pipeline, for example because you added or edited the file after a step had already run. The same list is part of the report from `MuscleBidsReport`.

Rarely needed settings for this part: see Per-subject file names in the appendix.

Analysis: the `analysis` section

The `analysis` section tells QMRITools which muscle values to measure and which pictures to make. It is the last step of the pipeline. It runs after your data is processed, merged and segmented, and it is started with `MuscleBidsAnalysis`.

The step writes:

- **Excel files (`.xlsx`)** with the values for each muscle, one file for each subject and session.
- **One combined Excel file**, named `All_<date>.xlsx`, with all subjects together.
- **Pictures (`.jpg`)** for quality control and presentations. You choose which ones with the `BidsOutputImages` option (see below).

All files go to the analysis folder (`05_analysis` by default).

Minimal analysis example

This measures the fat fraction from your Dixon data and the `md`, `rd` and `fa` values from your diffusion data, one value set per muscle. It also makes one fat fraction picture.

```
"analysis": {
  "Segmentation": {
    "Type": ["seg", "auto", "megre", "dix", "outph"],
    "Labels": [799, "Body"]
  },
  "Analysis": {
    "Types": [
      ["dwi", "dti", ["md", "rd", "fa"]],
      ["megre", "dix", ["fatfr"]]
    ]
  },
  "Images": {
    "Reference": ["megre", "dix", "wat"],
    "QuantImages": [
      [["megre", "dix", "fatfr"], "Lajolla", [0.0, 1.0], ["0", "100", "Fat Frac. [%]"]]
    ]
  }
}
```

How the section is built

The section has three parts:

- `Segmentation`: which muscle mask to use, and which muscles it contains.
- `Analysis`: which values to measure inside each muscle.
- `Images`: which pictures to make.

You can use the section in two layouts:

- **Flat** (the example above): the three parts sit directly under `analysis`. The results are saved without a block name.
- **Nested**: `analysis` contains one or more named blocks, and each block has its own three parts. See “Nested analysis blocks” below.

Segmentation

Key	What it is for	Default / allowed values
`Segmentation.Type`	The name parts of the segmentation file to measure in, for example `["seg", "auto", "megre", "dix", "outph"]`.	Required. No default.

Key	What it is for	Default / allowed values
`Segmentation.TypeSegmentation`	Optional. Measure some image types in their own segmentation, for example `{"dwi": ["seg", "auto", "dwi", "dti", "s0"], "mese": ["seg", "auto", "mese", "t2", "wat"]}`. Every entry of `Analysis.Types` whose name contains the image type (`dwi` , `megre` , `mese`) is measured in that segmentation, all others in `Type` . The muscle list, volume, cross-section and pictures always come from `Type` .	Not set: everything is measured in `Type` . If a muscle is missing in the other segmentation its values are 0 and the log tells which. If the file is missing or has another image size, the subject is skipped and the check file gets status `error` .
`Segmentation.Labels`	A number `n` , a word `what` , or both as `[n, what]` . The muscles are the labels up to `n` . Higher labels (bones and other structures) are not measured. `what` tells QMRITools which muscle names to use.	Required. For the default `Body` location use `799` , `"Body"` or `[799, "Body"]` . `what` is `"Body"` , `"Legs"` , `"Shoulder"` , `"Arm"` , or the path to your own label file. With any other text the muscles are named `Unknown` . A number alone uses `"Body"` . A word alone uses the number of the table below (`"Body"` 799, `"Legs"` 90, `"Shoulder"` 30, `"Arm"` 12), and a label file uses all labels of the segmentation.

Which `Labels` fits which `Segment.Location` :

`Segment.Location`	`Segmentation.Labels`	Muscles	Not analysed
`"Body"` (default) and the other names ending in `Body`	`[799, "Body"]`	labels 101 to 748	800 and up: bones and other structures, used for visualisation
`"Legs"` , `"LegsHip"` , `"UpperLegs"` , `"LowerLegs"` , `"Hip"` (older)	`[90, "Legs"]`	labels 1 to 90	101 and up: bones
`"Shoulder"` (older)	`[30, "Shoulder"]`	labels 1 to 30	101 and up: bones, vertebrae, ribs
`"Arm"` (older)	`[12, "Arm"]`	labels 1 to 12	21 and up: humerus

If `Labels` does not fit the segmentation, for example `[90, "Legs"]` or `"Legs"` for a `Body` segmentation, or is not a number, a word or both (for example `[1, 2]`), the analysis finds no muscle labels. It then skips that subject, writes a line in the log that tells which labels the segmentation has, and writes a check file with status `error` . The other subjects go on. Correct the `Labels` and run the analysis again; the subject is processed again because its check file is not `done` .

Analysis

Key	What it is for	Default / allowed values
`Analysis.Types`	The values to measure. Each entry is a list: dataset, suffix, then one or more maps. The last item is a list, so one entry can give several maps. `["dwi", "dti", ["md", "rd", "fa"]]` gives `md`, `rd` and `fa`.	Required.
`Analysis.TractBased`	Maps that are measured only inside the tracts, not in the whole muscle. Use it for diffusion values along the fibres. A map must also be listed in `Types` to be used.	Optional. Needs the tract density map (`dwi_dti_trk_dens`) from the tractography step.
`Analysis.MaskErosion`	If `true`, each muscle mask is made one voxel smaller at its edge. This leaves out edge voxels that mix muscle and fat or water.	Default `true`. Put this key directly under `Analysis`. An `Options` block inside `Analysis` is not read.

Each map gives two columns in the Excel file: the value and a spread value. The name is made from the last two parts of the entry. For example, `dix` and `fatfr` give `dix_fatfr` and `dix_fatfr_IQR`. The value is the median inside the muscle and the spread value is the interquartile range (IQR). For the tract maps `trk_seed` and `trk_dens`, the value is the mean and the spread value is the standard deviation, but the column is still named `..._IQR`.

Two maps are scaled to friendlier units in the Excel file. Fat fraction (`dix_fatfr` and `t2_fatfr`) is shown in percent (stored `0` to `1`, shown `0` to `100`). T2* from Dixon (`dix_t2star`) is shown in ms. All other maps keep the units they have in the files.

Tract-based maps always count each voxel once. There is no option to weight voxels by tract density.

Images

Key	What it is for	Default / allowed values
`Images.Reference`	A map used to place the 2D slices in the same position for every subject, and as background for the segmentation pictures.	Needed for the quantitative and segmentation pictures. If the file is missing, those pictures are skipped.
`Images.QuantImages`	Pictures of your maps. Each entry is `[[type], colour, [min, max], [legend min, legend max, legend text]]`.	Only the type: black and white, automatic range, no legend. The type and colour: automatic range. Add `[min, max]` to set the range.
`Images.TractImages`	A 3D picture of the tracts.	Optional. Needs the tract picture file (`dwi_dti_trk_plot`) from the tractography step.

In `QuantImages`, the `[min, max]` range is in the units of the map as it is stored. For fat fraction that is `0` to `1`. The legend text is only a label that is printed on the picture, so you can show `0` to `100` %. Without the fourth item there is no legend.

Without an `Images` block no pictures of your maps are made, and the segmentation pictures need `Images.Reference`. The segmentation pictures are made whenever a segmentation file exists and `BidsOutputImages` allows it. They are a 2D picture, a 3D picture (`_vol`) and a grid of slices (`_grid`).

Choosing the pictures: `BidsOutputImages`

The option `BidsOutputImages` decides which pictures are made. It is set when you run the analysis:

```
MuscleBidsAnalysis[dir, BidsOutputImages -> "Quantitative"]
```

Value	Pictures made
`"All"`	All pictures. This is the default.
`"Quantitative"`	Only the pictures of your maps (`QuantImages`).
`"Segmentation"`	Only the segmentation pictures.
`"Tractography"`	Only the tract picture (`TractImages`).
`"None"`	No pictures. Only the Excel files are made.

Output: the Excel files

There is one `.xlsx` file for each subject and session. It is saved in a folder for that subject and session inside the analysis folder. Each row is one muscle on one side.

Column	What it contains
`subject`, `subjectID`, `session`, `sessionID`	The subject and session.
`type`	The block name. Only in nested blocks.
`muscle`, `muscleID`	The muscle name and its label number.
`side`, `sideID`	`Left`, `Right` or `Both`, with `1`, `2` or `3`.
`volume`, `cross`	The volume of the muscle in cm^3 (ml), and its largest cross-section over the slices in cm^2.
`volume_F`, `cross_F`	Only with `UseFilter`: the volume and cross-section of the filtered part.
`dix_fatfr`, `dix_fatfr_IQR`, ...	One pair of columns for each map in `Types`.
`dix_fatfr_F`, `dix_fatfr_F_IQR`, ...	Only with `UseFilter`: the same pairs, measured only in the filtered part.

The combined file `All_<date>.xlsx` is in the same folder and holds the rows of all subjects.

Results for a file that already exists are not calculated again. To recalculate after you change the section, delete the `*_check.json` files in the analysis folder. Use `VersionCheck -> True` only after you update QMRITools. It redoes the results made with an older version.

Nested analysis blocks

Use a nested block when you want a separate Excel file and separate pictures for a different set of datasets, for example a second set of DTI data. Each block has a name. The name is added to the file names, so the blocks do not overwrite each other.

In a nested block, the first item of each list is the name of a dataset from the `datasets` section.

```
"analysis": {
  "OS": {
    "Segmentation": {
      "Type": ["DIXOS", "seg", "auto", "megre", "dix", "outph"],
      "Labels": [799, "Body"]
    },
    "Analysis": {
      "Types": [
        ["DIXOS", "megre", "dix", ["fatfr", "t2star"]]
      ]
    },
    "Images": {
      "Reference": ["DIXOS", "megre", "dix", "wat"],
      "QuantImages": [
        [["DIXOS", "megre", "dix", "fatfr"], "Lajolla", [0.0, 1.0], ["0", "100", "Fat Frac. [%]"]]
      ]
    }
  }
}
```

Add one more block name next to `"OS"` to make a second set of results. Each block works on its own.

Rarely needed settings for this part: see Analysis options in the appendix.

Checking and viewing your study

Before and during processing, you can look at your study in a few windows. They show you what the pipeline sees, which protocols are in your data, and which files have already been made. Nothing in these windows changes your data, except `ManageCheckFiles`, which can delete the check files described below, and `MuscleBidsGUI`, which runs the pipeline steps. `MuscleBidsReport` only writes a report file.

Several windows below show **check files**. A check file ends in `_check.json`. A step writes one next to its output when it has finished a subject, with the status `done`. `error` means an input was missing. For tractography, `track` and `seg` mean it is part of the way. A step skips what has a `done` check file (more below).

All functions take the folder that holds your `config.json` (your study folder, `dir` below). Run them in a notebook after loading the package and setting `dir`:

```
Needs["QMRITools`"]
dir = NotebookDirectory[];
```

ViewConfig

Why: to read your config file as tables, so you can see what each key is set to without opening the JSON file.

When: any time, for example after you edit `config.json`.

What you see: the sections of the config as tabs. Each tab is a table of keys and values. With a study folder, an extra tab called `default` shows the built-in settings, so you can compare your config with them.

```
ViewConfig[dir]
```

To look at the built-in settings only, use:

```
ViewConfig["default"]
```

SelectSubjects

Why: to pick subjects without typing their names. The names you pick are the names that `ProcessSubjects` and the subject lists below expect.

When: before you run a step on only some subjects.

What you see: a list of checkboxes with all subject folders from your `01_sourcedata` folder. Use **Select All** or **Deselect All**, or **Copy selected list to clipboard** to paste the list into your code, for example `{"001_1", "002_1"}`.

Use these folder names, not the `sub-...` names of the BIDS folders. A folder name without a session part (like `001`) is treated as session `001`.

```
SelectSubjects[dir]
```

ViewProtocolNames

Why: to see the names of the series that the scanner wrote. The `Label` values in your config must match these names exactly.

When: after `BidsDcmToNii` has put the raw files in `02_rawdata`, and before you write or change labels.

What you see: one menu entry per subject and session. Each entry is a table with the series number and protocol name. Rows in red are duplicate protocol names: two series have the same name. Tick **only problems** to show only the red rows. Press **Reload** after you change the data.

```
ViewProtocolNames[dir]
```

Only the folders in the config are needed, so this also works while you are still writing the labels.

CheckConfigLabels

Why: to find out whether each `Label` in your config is really present in each session. A missing label means that data will not be converted.

When: after `BidsDcmToNii`, because it reads the subjects from `02_rawdata`.

What you see: one menu entry per subject and session. Each row is a dataset, its label, and `yes` or `missing`. Red rows are labels that were not found. Use **only problems** to show only those, and **Reload** after you change anything.

```
CheckConfigLabels[dir]
```

To check only some subjects, give them as a second argument:

```
CheckConfigLabels[dir, {"001_1", "002_1"}]
```

CheckBidsOutput

Why: to check that the converted files (the `.json` and `.nii` files of each label) are in the `02_rawdata` folder of the subject after the conversion step.

When: after `MuscleBidsConvert`.

What you see: one menu entry per subject and session. Each row is a dataset, its label, the converted file, and `yes` or `missing`. Red rows have no `.json` file (`no json found`), no matching `.nii` file (`missing`), or a `.json` file with 0 bytes (`json empty`, a conversion that stopped while writing), so that label was not converted correctly. It only checks that the label was converted, not that every contrast of the dataset is complete. For a dataset of class `Volumes`, which joins several labels into one file, only the first label is checked.

```
CheckBidsOutput[dir, "Convert"]
```

Only `"Convert"` is available as a step.

CheckCustomConfigs

Why: to see which subjects have a per-subject config, what it changes, and whether the pipeline picked it up.

When: after you add or edit a custom config, and before you run a step.

What you see: a menu of the subjects that have a per-subject config. Each entry is a subject, and each row is one setting that the subject changes: the dataset, the setting (nested keys are joined with a dot, for example `Merging.Overlap`), the value of the subject and the value in your study config. A dataset that is not in your study config is marked `(not in config)`. A subject without a custom config is not listed.

A custom config starts in the source folder and every step copies it along with the data, so the pipeline keeps using the same settings. The check compares these copies for each subject, from the source folder to `02_rawdata`, `03_derivatives` and `04_merged`. A red row means that a later copy is `not copied` (the folder of that step exists but has no copy), `unreadable`, or `differs from` the first copy. This normally means that the custom config was added or changed after that step ran. Run the steps again for that subject (delete their check files first, see How to force a redo) so the new settings are used. Only subjects that already have a folder in `02_rawdata` are listed. Every table scrolls when it has more than 15 rows. The scan time is shown, and **Reload** scans again.

```
CheckCustomConfigs[dir]
```

The option `ProcessSubjects` limits the subjects, as for the other checks.

MuscleBidsConfig

Why: to open the config and all checks on the config in one window, so you do not have to call each function.

When: whenever you write or change the config, or want to know which subjects have a custom config.

What you see: four tabs: **Full config**, **Custom config**, **Config labels** and **Protocol names**. **Full config** is `ViewConfig`, **Custom config** is `CheckCustomConfigs`, **Config labels** is `CheckConfigLabels` and **Protocol names** is `ViewProtocolNames`.

```
MuscleBidsConfig[dir]
```

The option `ProcessSubjects` limits the subjects, as for the other checks.

MuscleBidsCheck

Why: to open the checks of the processing steps in one window.

When: whenever you want an overview of what has been processed, for example before you start a run.

What you see: three tabs: **Convert output**, **Check files** and **Report**. **Convert output** is `CheckBidsOutput`, **Check files** is `ManageCheckFiles` and **Report** is described under MuscleBidsReport. The checks of the config are in MuscleBidsConfig.

```
MuscleBidsCheck[dir]
```

To look at some subjects only:

```
MuscleBidsCheck[dir, ProcessSubjects -> {"001_1"}]
```

If you ran the analysis without images (`BidsOutputImages -> "None"`), give the same option here, otherwise the **Check files** tab expects image check files that were never made.

ManageCheckFiles

Why: to see which processing steps have already finished for each subject, and to delete their check files if you want them run again.

When: when a step did not run and you want to know why, or when you want to force a step to run again.

What you see: one tab per processing step: **Process**, **Merge**, **Segment**, **Tractography** and **Analysis**. Each tab lists the check files that the step expects for each subject, with a status (`done`, `track`, `seg` or `error`), the version of QMRITools that made it, and the date. A file that does not exist yet is shown as `missing` in red, and a file that cannot be read as `unreadable` in red. Within a step there is a subtab per data type, and when there is more than one data type, a last **All** subtab that lists all check files of that step together.

- Tick a file to select it (only existing files have a tick box), or use **Select all** and **Deselect all**.
- **Delete selected** removes the selected check files of that tab after you confirm. The next run of that step then processes that data again.
- **Reload** rescans the study. Use it after you add subjects or datasets.
- **Refresh status** reads the known check files again, without searching for new subjects or datasets. It is much faster than **Reload** on a large study. Use it after a step has run.

If you ran the analysis with `BidsOutputImages -> "None"`, give the same option here (see `MuscleBidsCheck`).

```
ManageCheckFiles[dir]
```

MuscleBidsReport

Why: to get a list of everything that is wrong in your study, as a file you can read, print or send to a colleague, without opening the windows above.

When: after a run, or before you hand over the study. It works from a script, so you can also call it without opening any window. The **Report** tab of `MuscleBidsCheck` and `MuscleBidsGUI` does the same with buttons.

What you get: one file with the problems of one step or of all steps. The file starts with the study folder, the date, the QMRITools version, the number of subjects and the datasets. Then comes a summary table with the number of problems per step, and the problems themselves in tables. They are grouped by kind of problem:

Kind	Meaning	Steps
`error`	The step ran and wrote a check file with the status `error`, with its date	Process, Merge, Segment, Tractography, Analysis
`unreadable`	The check file exists but cannot be read	Process, Merge, Segment, Tractography, Analysis
`missing`	A label, a converted file or a check file that should exist is not there	Config labels, Convert, Process, Merge, Segment, Tractography, Analysis
`duplicate`	Two series have the same protocol name	Protocol names
`outdated`	A custom config of a subject was not copied to a later step, or differs from the first copy	Custom configs

The `unreadable` kind also covers a custom config file that cannot be read. The report of **Custom configs** (and the report of all steps) also has an information section, **Custom configs (n settings)**, that lists every setting that a subject changes. This section is not counted as problems, and the header of the report shows how many subjects have a custom config.

For a report of all steps the tables are grouped by kind first and then by step. For one step they are grouped by kind. Each problem of a processing step and of Convert has the path of the log file of that subject (relative to the study folder, when the file exists), because a check file with the status `error` does not say what went wrong. The reason is in that log. A step that is not checked, for example because `config.json` cannot be read, is mentioned in the report and is never reported as "No problems found".

```
(* All steps, as a pdf *)
MuscleBidsReport[dir, "All", FileNameJoin[{dir, "report.pdf"}]]

(* One step, as markdown *)
MuscleBidsReport[dir, "Merge", FileNameJoin[{dir, "report_merge.md"}]]
```

The steps are `"Protocol names"`, `"Config labels"`, `"Convert"`, `"Custom configs"`, `"Process"`, `"Merge"`, `"Segment"`, `"Tractography"` and `"Analysis"`, or `"All"`. The extension of the file name selects the format, `.md` (markdown) or `.pdf`. The folder of the file is made when it does not exist, and the function returns the path of the file, or `$Failed` with a message (for example for a wrong step name or file extension).

The options are the same as for `MuscleBidsCheck`: `ProcessSubjects` for some subjects only, and `BidsOutputImages -> "None"` when you ran the analysis without images.

```
MuscleBidsReport[dir, "All", FileNameJoin[{dir, "report.md"}], ProcessSubjects -> {"001_1", "002_1"}]
```

The Report tab. In `MuscleBidsCheck` and `MuscleBidsGUI` the **Report** tab has a choice of the format (`md` or `pdf`), the output folder (with a **Browse** button; the study folder is the default), a button **Export all steps** and one button per step. Each button scans the study again, so the report is current, and writes `<study>_report_<step or all>_<date>.<md or pdf>` to the folder. The status line shows the file, and **Open** opens it.

MuscleBidsGUI

Why: to run the pipeline steps from one window, without typing the commands, and to follow the progress.

When: when you want to run steps with the settings of your `config.json` and look at the checks in the same place.

What you see: three tabs.

- **Run**: choose the subjects (**All subjects**, or untick it and pick subjects as in `SelectSubjects`). Each step of the pipeline has a button, with its own options beside it: `DeleteAfterConversion` for `MuscleBidsConvert`, `VersionCheck` for the processing steps, `BidsTractographyMethod` for `MuscleBidsTractography` and `BidsOutputImages` for `MuscleBidsAnalysis`. The options start at the default of the step. Below are the status (a moving bar, the state, the time the step is running, and how many subjects have started), a **Log** with the last lines of the log files, and the **Messages** of the step.
- **Config**: the same as `MuscleBidsConfig[dir]`: the full config, the custom configs, the config labels and the protocol names.
- **Check**: the same as `MuscleBidsCheck[dir]`: the convert output, the check files and the **Report** tab.

The **Config** and **Check** tabs do not scan the study when the window opens, so on a large study the window appears at once. Open a subtab and press **Reload** to scan it the first time. Until then it says `Not scanned yet, press Reload`. On the **Check files** subtab, **Refresh status** only reads the check files that are already known, which is much faster than **Reload**. Use it after a step has run. Use **Reload** after you add subjects or datasets.

```
MuscleBidsGUI[dir]
```

Each step runs in a separate Mathematica kernel. The notebook stays free and nothing is printed in it. Only one step runs at a time, and the step buttons are disabled while a step runs. The state is `Running`, then `Done`, `Done with messages` (the step ended, but wrote warnings; read the **Messages**), `Crashed` (the kernel could not be started or stopped), or `Stopped`. A window that is shown again while a step was running shows `Interrupted`. **Stop** ends the step after you confirm. For the steps that use check files, the subjects that were not finished are processed again the next time you run the step, because a step only writes its check file when a subject is finished. Keep the notebook with the window open while a step runs.

The window always runs the steps on the folder you gave it, with the subject choice made in the **Run** tab. To use the other options of a step, for example the `ProcessSubjects` of a script, call the step in a notebook instead.

How the pipeline skips finished work

Each finished step writes a small file that ends in `_check.json` next to its output. Before a step starts on a subject, it looks for this file. If the file exists and says the step is done, the subject is skipped. This is why running a pipeline twice does not repeat the work.

The conversion step (`MuscleBidsConvert`) does not use check files. It converts every raw file it finds. By default (`DeleteAfterConversion -> True`) it deletes the raw files after converting them, so a second run finds nothing to do.

The version check works like this:

- `VersionCheck -> False` (the default) skips finished data, whichever version of QMRITools made it.
- `VersionCheck -> True` processes finished data again if it was made with an older version of QMRITools.

How to force a redo

Changing `config.json` does not start a redo by itself. Delete the check files of the changed step **and of all later steps**, then run those steps again. There are two ways to make a step run again:

1. **Delete the check files.** Open `ManageCheckFiles`, select the files of that step, and click **Delete selected**. Or delete the `*_check.json` files in your file browser.
2. **Use** `VersionCheck -> True` after you update QMRITools. Only data made with an older version is processed again.

```
MuscleBidsProcess[dir, VersionCheck -> True]
```

`BidsDcmToNii` skips a subject when its `raw` folder is not empty. To redo that conversion, empty the `raw` folder of the subject and session (`02_rawdata/sub-.../ses-.../raw`) and run `BidsDcmToNii` again.

Log

The pipeline writes what it does to a log while it runs, and opens the log window when a step starts. Each subject and step also has a `.log` file in its output folder (for example `sub-001_ses-1_BIDSProcess.log`). If you closed the log window, open it again with:

```
ShowLog[]
```

Nothing happened: what to check

Use this list when a step did nothing, or skipped a subject you expected to be processed.

- **A label does not match the protocol name.** Open `ViewProtocolNames` and `CheckConfigLabels`. A missing label is red. Copy the protocol name exactly, including capitals and spaces, into `Label`.
- **Two series have the same protocol name.** Duplicates are red in `ViewProtocolNames`. Check the series numbers.
- **The raw files are missing.** Open `CheckBidsOutput`. A label with `missing` or `no json found` was not converted. Check that the subject is in `02_rawdata` and that `BidsDcmToNii` ran.
- **A converted `.json` file is empty.** `CheckBidsOutput` shows `json empty` when the `.json` of a label has 0 bytes. This happens when `MuscleBidsConvert` stopped while it wrote the file (the end of the `BIDSConvert.log` of the subject shows where). Delete the empty `.json` and its `.nii` file, and convert the subject again. This needs the raw files, so check that they are still in the `raw` folder (`DeleteAfterConversion -> True` removes them after a conversion).
- **The subject name in `ProcessSubjects` does not match.** If no subject matches, nothing is processed and no error is shown (the views show "No subjects found"). Use the folder names from `SelectSubjects`.
- **A custom config is not used.** A subject with its own `config.json` that you added or changed after a step ran keeps the old settings in that step. Run `CheckCustomConfigs`: a red row (`not copied` or `differs from`) shows where the copy of the config is wrong. Delete the check files of that subject and run the steps again.
- **An old check file is still there.** Open `ManageCheckFiles`. A `done` file makes the step skip that subject. Delete it to run the step again.

- **The config has an error.** If `config.json` cannot be found or read, the views use the default folder names and show no datasets. Datasets that fail the config check are left out. Open `ViewConfig` to check the file.
- **A dataset has no `Label`.** Such a dataset gives no rows in `CheckConfigLabels` and `CheckBidsOutput`.
- **A file name does not match.** Most "nothing happened" cases are this: the step looks for a file with a specific name, and that file is not on disk. Set `QMRITools`$Debug = "Bids"` to print the expected path, compare it with the files in the folder, then set it back to `False`.

Appendix: advanced settings

These settings are rarely needed. The defaults suit most studies. They are grouped by the part of the manual they belong to.

Default config

Every key you leave out of `config.json` gets the value below. The blocks `Process`, `Merging`, `Segment`, `Tractography` and `Analysis` are the defaults of the blocks with the same name in a dataset (or in `analysis`). In the notebook, `ViewConfig["default"]` shows the same values.

```
{
  "folders": {
    "dicomData": "01_sourcedata",
    "rawData": "02_rawdata",
    "derivedData": "03_derivatives",
    "mergeData": "04_merged",
    "analysis": "05_analysis"
  },
  "conversion": {
    "Version": "Own"
  },
  "Process": {
    "Masking": 5,
    "FlipPermute": [[1, 1, 1], ["x", "y", "z"]],
    "SplitRegistration": false,
    "RegistrationDimension": "3D",
    "RegistrationMethod": "bspline",
    "RicianCorrection": true,
    "IVIMCorrection": true,
    "GradientCorrection": false,
    "FasciculationDetection": false
  },
  "Merging": {
    "Overlap": 0,
    "Padding": 0,
    "Motion": true,
    "Reverse": false,
    "SplitRegistration": true
  },
  "Segment": {
    "Device": "CPU",
    "Dimensions": "3D",
    "Location": "Body"
  },
  "Tractography": {
    "FlipPermute": [[1, 1, 1], ["x", "y", "z"]],
    "TractLength": [15, 500],
    "TractAngle": 25,
    "TractSeed": 0.66,
    "TractStep": "Automatic",
    "SegmentLength": [15, 500],
    "BoneLabel": 100,
    "HarmonicDenoise": false
  },
  "Analysis": {
    "MaskErosion": true,
    "UseFilter": "None"
  }
}
```

Notes:

- `conversion.Version` is `"Own"` on Windows and `1` on macOS.
- `Segment` also has the settings `VoxSize` and `Method`. They are not set by default, so they are left out above.
- Settings without a default (`Label` , `Type` , `Suffix` , `Process.Method` , `Merging.Target` , `Merging.Moving` , `Merging.Process` , `Segment.Target` , `Tractography.Target` , `Tractography.Stopping` , `Settings` for `EPGT2`) must be written in your config. This section is only a reference: do not copy it as your `config.json` , because the blocks belong inside the datasets.

Pipeline step options

Option	Step	What it is for	Default / allowed values
`DeleteAfterConversion`	`MuscleBidsConvert`	Delete the raw files after they are converted. Use `False` to keep them.	`True` ; `True` or `False`
`BidsTractographyMethod`	`MuscleBidsTractography`	Run only the tracking, or only the splitting of the tracts per muscle (needs an existing tract file).	`"Full"` ; `"Full"` , `"Tractography"` or `"Segmentation"`
`BidsOutputImages`	`MuscleBidsAnalysis`	Which images to make. `"None"` makes only the Excel files.	`"All"` ; `"All"` , `"Quantitative"` , `"Segmentation"` , `"Tractography"` or `"None"`

For example, to keep the raw files:

```
MuscleBidsConvert[dir, ProcessSubjects -> All, DeleteAfterConversion -> False]
```

Dataset options

Key	What it is for
`Label` as a list with `Class` `"Volumes"`	Only the first label is used in the file name, all labels are joined.
`Suffix` of a merge target	A `Merging` `Target` without a key finds its dataset through the suffix. Keep the `Suffix` of a Dixon dataset equal to the folder name `dix` .

Process options

Change these only when the defaults do not fit your data.

Key	What it is for	Default / allowed values
`EchoTime`	Echo times in ms. `Dixon-B` needs `[first echo, echo spacing]` . For `mese` scans (`EPGT2` , `Exp`) a single number is the echo spacing (the first echo equals the spacing). Giving it for a `mese` scan means the data is read as one 4D file, so use it only when the echo times are not stored correctly in the scan. This only applies to data converted with an older `dcm2niix` . The `dcm2niix` that comes with QMRITools writes one file per echo with the echo times in the json files, so the setting is then ignored (and the log says so).	none
`Masking`	Threshold for the mask that removes the background. Used by `DTI` , `EPGT2` and `Exp` .	`5`
`FlipPermute`	For `DTI` . Flips or swaps the axes of the gradient directions to match your scanner. Format `[[flipX, flipY, flipZ], [axis1, axis2, axis3]]` , with flips `1` or `-1` .	`[[1,1,1],["x","y","z"]]`
`RegistrationDimension`	For `DTI` . `"2D"` registers each slice on its own (for example for cardiac data) and turns off `SplitRegistration` .	`"3D"`
`RegistrationMethod`	For `DTI` . The method used to correct motion and distortions between the diffusion volumes.	`"bspline"` . Also `"translation"` , `"rigid"` , `"affine"`
`SplitRegistration`	For `DTI` . Registers the left and right side of the muscle separately.	`false`
`FasciculationDetection`	For `DTI` . Also detects fasciculations (spontaneous muscle twitches) in the diffusion data and adds the `fasc` , `fascm` and `norm` maps.	`false`

Merging options

Key	What it is for	Default / allowed values
`Padding`	Pads the overlap region with this many slices. Has no effect when `Overlap` is `0` .	`0`

Key	What it is for	Default / allowed values
`Motion`	Set to `false` to skip the motion correction between the chunks, and the alignment of the chunks when the `Target` is the same map as `Moving`.	`true`
`Reverse`	Use when the chunks were acquired in the reverse order, so that they are stacked from the other end.	`false`
`SplitRegistration`	Registers all chunks after the first one separately for the left and right side.	`true`

Segment options

Key	What it is for	Default / allowed values
`Location` (other values)	Region-specific sets of the older models. Names ending in `Body` (`"LegsBody"`, `"LegsHipBody"`, `"HipBody"`, `"UpperLegsBody"`, `"LowerLegsBody"`, `"ShoulderBody"`) are parts of the full-body model. The older sets are `"Legs"`, `"LegsHip"`, `"UpperLegs"`, `"LowerLegs"`, `"Hip"`, `"Shoulder"` and `"Arm"`. The `Labels` to use in the analysis are in Segmentation.	`"Body"`
`Dimensions`	`"2D"` segments each slice on its own. `"3D"` uses the whole volume at once. Any other value is read as `"3D"`.	`"3D"`
`Device`	Where the neural network runs. Use `"GPU"` only when your computer has a suitable graphics card.	`"CPU"`
`Method`	Leave it out (do not write `"Automatic"`). The experimental value `"Registration"` aligns an existing segmentation to your data instead of using the network. It uses the keys `Target`, `Moving`, `Segmentation` and `VoxSize` (voxel size to resample to).	not set

Tractography options

Key	What it is for	Default / allowed values
`TractLength`	Shortest and longest tract, in mm. Shorter tracts are removed, longer tracts are cut.	`[15, 500]`
`TractAngle`	Largest angle change allowed per step of a tract, in degrees.	`25`
`TractSeed`	How many starting points (seeds) are used. A value below 1 is a fraction of the possible starting points. A value of 1 or more is the number of seeds.	`0.66`
`TractStep`	Length of each tracking step in mm. Any text, such as `"Automatic"`, lets the code choose. A number sets the step length.	`"Automatic"`
`SegmentLength`	Shortest and longest tract (mm) that is kept when the tracts are split per muscle.	`[15, 500]`
`BoneLabel`	Label number that separates muscles from bones in the segmentation. Labels `1` up to this number are muscles. The next 30 labels are bones (only used to draw the 3D picture of the tracts).	`100`
`HarmonicDenoise`	Set to `true` to denoise the tensor with a harmonic filter before the tracts are split per muscle. Needs `Segmentation`. It creates extra files.	`false`
`FlipPermute`	Flips or swaps the axes of the tensor when the orientation of your scanner differs from the standard. Format `[[flipX, flipY, flipZ], [axis1, axis2, axis3]]`. It is a separate setting from `FlipPermute` in `Process`, which corrects the gradient directions before the tensor is fitted.	`[[1,1,1],["x","y","z"]]`

Per-subject file names

Key	What it is for	Default / allowed values
`sub-<subject>_ses-<session>_config.json`	Same as `config.json`, but with the subject and session in the name, for example `sub-001_ses-2_config.json`. `config.json` is used first if both exist.	none

Analysis options

Key	What it is for	Default / allowed values
`Analysis.Class`	Set to `"Stacks"` if your datasets are stacks. This sets the `stk` part of the file names. For other data, the `chunk` part is used. Set it here, under `Analysis`.	Default: `chunk`.
`Analysis.UseFilter`	Adds the `_F` columns. Only voxels whose value in a second map is inside the range are used for them. Example: `[[0.0, 0.2], ["megre", "dix", "fatfr"]]`.	Default `"None"`. Otherwise `[[min, max], [map]]`, in the units of that map.

Key	What it is for	Default / allowed values
`Segmentation.Labels`, second item	The path to your own label file, instead of `"Body"`, `"Legs"`, `"Shoulder"` or `"Arm"`.	An ITK-SNAP label description file: one line per label, starting with the label number and ending with the muscle name in quotes. Lines starting with `#` are ignored. A name ending in `Left` or `Right` (for example `"Vastus lateralis Left"`) sets the side; other names get the side `Both`.